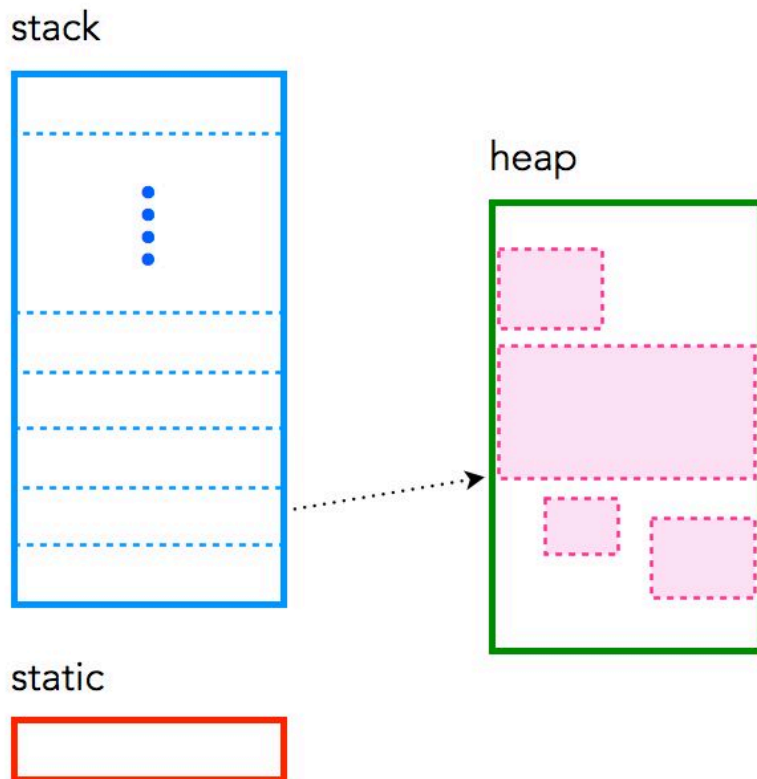


The Craft of Coding

Memory in C – the stack, the heap, and static

The great thing about C is that it is so intertwined with memory – and by that I mean that the programmer has quite a good understanding of “*what goes where*”. C has three different pools of memory.

- **static**: global variable storage, permanent for the entire run of the program.
- **stack**: local variable storage (automatic, continuous memory).
- **heap**: dynamic storage (large pool of memory, not allocated in contiguous order).



(<https://craftofcoding.wordpress.com/wp-content/uploads/2015/12/stackmemory4.jpg>).

Static memory

Static memory persists throughout the entire life of the program, and is usually used to store things like *global* variables, or variables created with the **static** clause. For example:

```
int theforce;
```

On many systems this variable uses 4 bytes of memory. This memory can come from one of two places. If a variable is declared *outside* of a function, it is considered global, meaning it is accessible anywhere in the program. Global variables are **static**, and there is only one copy for the entire program. Inside a function the variable is allocated on the stack. It is also possible to force a variable to be static using the **static** clause. For example, the same variable created inside a function using the **static** clause would allow it to be stored in static memory.

```
static int theforce;
```

Stack memory

The *stack* is used to store variables used on the inside of a function (including the **main()** function). It's a LIFO, "Last-In,-First-Out", structure. Every time a function declares a new variable it is "pushed" onto the stack. Then when a function finishes running, all the variables associated with that function on the stack are deleted, and the memory they use is freed up. This leads to the "local" scope of function variables. The stack is a special region of memory, and automatically managed by the CPU – so you don't have to allocate or deallocate memory. Stack memory is divided into successive frames where each time a function is called, it allocates itself a fresh stack frame.

Advertisement

Note that there is generally a limit on the size of the stack – which can vary with the operating system (for example OSX currently has a default stack size of 8MB). If a program tries to put too much information on the stack, **stack overflow** will occur. Stack overflow happens when all the memory in the stack has been allocated, and further allocations begin overflowing into other sections of memory. Stack overflow also occurs in situations where recursion is incorrectly used.

A summary of the stack:

- the stack is managed by the CPU, there is no ability to modify it
- variables are allocated and freed automatically
- the stack is not limitless – most have an upper bound

- the stack grows and shrinks as variables are created and destroyed
- stack variables only exist whilst the function that created them exists

Heap memory

The *heap* is the diametrical opposite of the stack. The *heap* is a large pool of memory that can be used dynamically – it is also known as the “free store”. This is memory that is not automatically managed – you have to explicitly allocate (using functions such as `malloc`), and deallocate (e.g. `free`) the memory. Failure to free the memory when you are finished with it will result in what is known as a *memory leak* – memory that is still “being used”, and not available to other processes. Unlike the stack, there are generally no restrictions on the size of the heap (or the variables it creates), other than the physical size of memory in the machine. Variables created on the heap are accessible anywhere in the program.

Oh, and heap memory requires you to use **pointers**.

A summary of the heap:

- the heap is managed by the programmer, the ability to modify it is somewhat boundless
- in C, variables are allocated and freed using functions like `malloc()` and `free()`
- the heap is large, and is usually limited by the physical memory available
- the heap requires pointers to access it

An example of memory use

Consider the following example of a program containing all three forms of memory:

```
#include <stdio.h>
#include <stdlib.h>

int x;

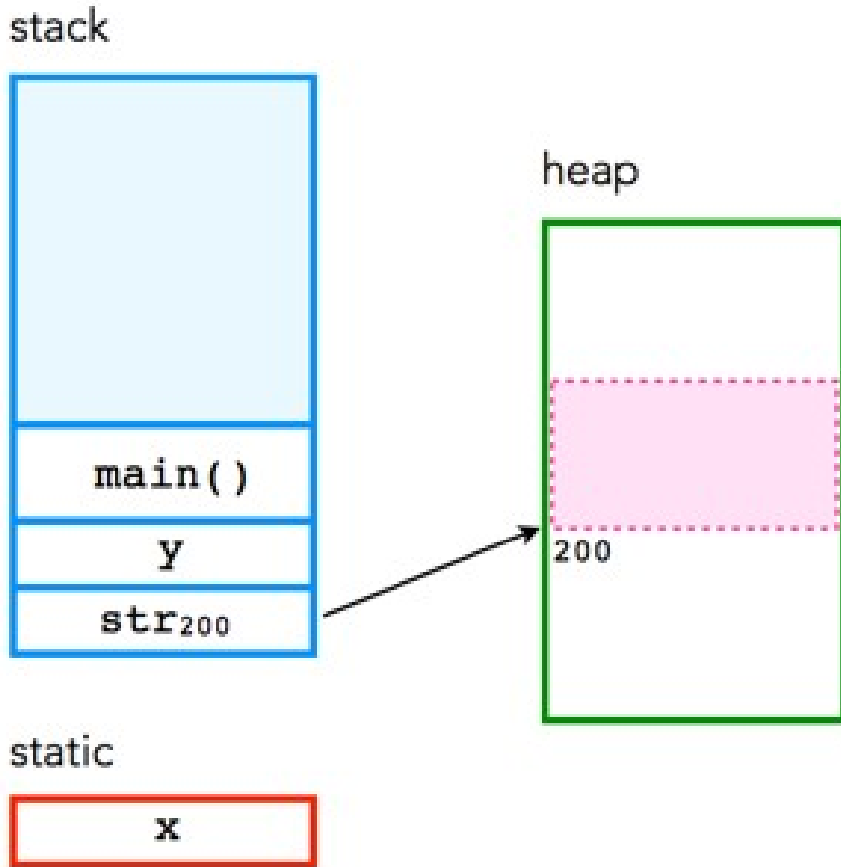
int main(void)
{
    int y;
    char *str;

    y = 4;
    printf("stack memory: %d\n", y);

    str = malloc(100*sizeof(char));
    str[0] = 'm';
    printf("heap memory: %c\n", str[0]);
    free(str);
    return 0;
}
```

The variable **x** is static storage, because of its global nature. Both **y** and **str** are dynamic stack storage which is deallocated when the program ends. The function **malloc()** is used to allocate 100 pieces of of dynamic heap storage, each the size of char, to **str**. Conversely, the function **free()**, deallocates the memory associated with **str**.

Advertisement



(<https://craftofcoding.wordpress.com/wp-content/uploads/2015/12/stackmemory31.jpg>).

Advertisement

📅 07/12/201508/09/2018

👤 spqr

🔧 automatic

🔧 heap

🔧 stack

22 thoughts on “Memory in C – the stack, the heap, and static”

1. **Martin Muguna** says:

16/02/2018 at 7:01 pm

Wooow.

You made that so simple to understand

Great work

Definitely following this blog

Thanks

[↩ Reply](#)

2. **Deepika** says:

19/02/2018 at 6:46 am

woww!!!

I can understand the program easily!!

thanks!

surely following this blog!!

[↩ Reply](#)

3. **aade** says:

02/04/2018 at 1:25 pm

very nice explanation. keep up the good work

[↩ Reply](#)

4. **sbkrish** says:

08/09/2018 at 2:06 am

Explanation about memory allocation is really nice. But I have a doubt in stack memory storage (in the image). It is LIFO concept, and it will be accessed from the top. In the image the order will be, y, str100, main().

[↩ Reply](#)

1. **spqr** says:

08/09/2018 at 2:24 am

The order should be main(), y, str100... I have fixed the figure... and although it is a stack, it could be accessed from either top or bottom.

[↩ Reply](#)

5. Pingback: [スタックとヒープを知る | mrsekutの備忘録](#)

6. **OmarOuali@outlook.com** says:

29/11/2018 at 5:33 pm

Thank you

[↩ Reply](#)

7. Pingback: [Memory Model – Clover I/O](#)

8. **shooketh** says:

30/09/2019 at 6:17 am

Great explanation! Thanks!

[↩ Reply](#)

9. **Partha** says:

03/02/2020 at 1:59 am

AN EXAMPLE OF MEMORY USE – that one section cleared all my confusions. The image & explanations are simply outstanding. I have bookmarked this page under the permanent section. Thanx for this great article!

[↩ Reply](#)

10. Pingback: [I am learning C in 2020 – Beginner - What I learned about memory, variables and pointers in c - Comfort Ajala](#)

11. **Funnymouth** says:

16/06/2020 at 12:50 am

Really good explanation!

Moving from high level languages to learning C, I was confused about memory management, this one article cleared up so many things for me.

[↩ Reply](#)

12. **Jiv** says:

22/07/2020 at 9:09 pm

Nice explanation.

3 Questions from a C beginner:

1. In your example if I declare y as:

```
static int y;
```

instead of the way you did in your example, will y be stored in the static, stack or heap area of memory?

2. If main calls a function named func1, in which I declare a variable z as:

```
static int z;
```

will z be stored in the static, stack or heap area of memory?

3. Will the answer to Question 2 remain unchanged if func1 calls another function named func2 and func1 passes z to func2 by address, i.e., &z?

[↩ Reply](#)

1. **spqr** says:

23/07/2020 at 8:06 pm

Statics are stored in static space, as they persist.

[↩ Reply](#)

1. **Jiv** says:

23/07/2020 at 8:34 pm

Thanks SPQR. A related question: Since I work with numerical algorithms with 2-dimensional matrices that can get as large as 10,000 x 10,000 of type double, i.e., almost 1 GB in size, are there any limits on the size of memory available in the static space, assuming physical memory available is 16 GB, and I am using 64 bit Windows with Visual Studio 2013 C with 64 bit configuration?

2. **spqr** says:

31/07/2020 at 1:55 pm

Likely based on the OS you are using. I wouldn't necessarily go too large with static arrays in C.

13. **jivkalehotmailcom** says:

28/08/2020 at 7:35 pm

C++ string memory allocation question:

I understand that a variable declared as string is stored on the stack if it has less than 24 characters, but if the string is longer than 24 characters, then memory for it is allocated on the heap.

Where is an array of string stored, if it is declared in the body of the main program as:

```
std::array assetid;
```

when the length of the string in each element of the array is less than 24 characters?

Also, given that the length of the string in each element of the array is not specified during declaration, where is the array stored if it is declared in the body of the main program as:

```
static std::array assetid;
```

when the length of the string in each element of the array is less than 24 characters, or if it is more than 24 characters?

[↪ Reply](#)

1. **spqr** says:

04/09/2020 at 12:53 am

Hi,

Sorry, C++ memory is likely a little different to C, and therefore I can't really answer the question.

[↪ Reply](#)

14. **Thomas** says:

24/03/2021 at 10:20 am

what about `char *str2 = "hello world!"` inside main method??? It's not in the heap?

[↪ Reply](#)

1. **JimB** says:

10/07/2021 at 7:17 am

Thomas, this is actually 2 separate bits of memory. The pointer "str2" would be stored on the stack. However the constant string "hello world" (which the pointer will point to) is static. Note though that "static" can actually be further divided into read/write or read-only. A constant string would normally be in a read-only chunk of static memory. So if you tried to do:

```
str2[0] = 'x'; // try to overwrite the 'h' in "hello world"
```

This would result in a memory exception as "hello world" is in write protected memory.

[↪ Reply](#)

15. **kedari Abhilash** says:

26/03/2022 at 6:25 am

Great Explanation

[↪ Reply](#)

16. **Debra O** says:

07/01/2025 at 9:30 am

Very nice blog you have here

[↪ Reply](#)

This site uses Akismet to reduce spam. [Learn how your comment data is processed.](#)

BLOG AT WORDPRESS.COM.